



AuditWare

X

0xbow

Smart Contract Security Review

Performed by **Auditware** for **0xbow** on **2/28/2025**



auditware.io



audit_wizard



Review Summary

The Auditware team has performed a security review on 0xbow's Privacy Pools protocol. We restricted the scope to the portions that provide private asset transfers and allow users to publicly deposit funds and make partial private withdrawals, so long as they can prove membership in an approved set of whitelist addresses.

The scope consisted of the following contracts:

- src/contracts/Entrypoint.sol
- src/contracts/PrivacyPool.sol
- src/contracts/State.sol
- src/contracts/implementations/PrivacyPoolComplex.sol
- src/contracts/implementations/PrivacyPoolSimple.sol
- src/contracts/lib/Constants.sol
- src/contracts/lib/ProofLib.sol

Audit commit hash: [f0b8fe5](#)

The focus of the audit was to evaluate potential security risks associated with the programs in scope, looking to identify vulnerabilities and recommend measures to mitigate identified risks.

Although best practices were applied and the contract was written with security in mind, medium and low issues were found. No critical or urgent concerns were found. Detailed explanations for each item can be found in the report.

As with all smart contracts, it is highly recommended to:

- Conduct regular and thorough audits to identify and mitigate vulnerabilities.
- Achieve the highest possible test coverage, ideally approaching 100%.
- Maintain comprehensive and up-to-date documentation.
- Participate in auditing contests and/or a bug bounty program.
- Implement on-chain monitoring for real-time threat detection and mitigation.



Findings Overview

Finding	Type	Risk Level	Status
<u>AW-M-01: Insecure ERC20 Token Approval Handling</u>	Unexpected Behavior	Medium	Mitigated
<u>AW-M-02: Potential DoS Due to Root History Overwrite in Withdrawal Mechanism</u>	DoS	Medium	Partially Mitigated
<u>AW-M-03: Unaudited Poseidon Hash Function Library</u>	3rd Party	Medium	Acknowledged
<u>AW-L-01: DoS Risks in ASP Root Management and Withdrawal Processing</u>	DoS	Low	No Risk (by design)
<u>AW-L-02: Incorrect Index Returned in updateRoot Function</u>	Unexpected Behavior	Low	Mitigated
<u>AW-L-03: Unbounded Relayer Fee</u>	Unexpected Behavior	Low	Mitigated



AW-M-01: Insecure ERC20 Token Approval Handling

Severity: *Medium*

Status: *Mitigated*

Code:

- [packages/contracts/src/contracts/Entrypoint.sol#L201-L201](#)
- [packages/contracts/src/contracts/Entrypoint.sol#L216-L216](#)

Description:

During the audit we found usages of `approve` without recommended handling.

For tokens like USDT, which require setting the allowance to zero before updating it to a new value, the current implementation can lead to unexpected reverts.

Additionally, USDT on Ethereum mainnet lacks a boolean return value for the `approve` method, which means that the Solidity call to USDT will revert when attempting to decode the return value.

Recommendations:

- Use `forceApprove` from openzeppelin's SafeERC20 contract to ensure correct handling of all ERC20 tokens.

```
- if (address(_asset) != Constants.NATIVE_ASSET) _asset.approve(address(_pool),  
type(uint256).max);  
  
+ if (address(_asset) != Constants.NATIVE_ASSET) _asset.forceApprove(address(_pool),  
type(uint256).max);
```

Mitigation:

<https://github.com/0xbow-io/privacy-pools-core/blob/ac36feba8ba26f3485acd0d87a9d5c373ab3967d/packages/contracts/src/contracts/Entrypoint.sol#L209-L224>



AW-M-02: Potential DoS Due to Root History Overwrite in Withdrawal Mechanism

Severity: *Medium*

Status: *Partially Mitigated*

Code:

- [packages/contracts/src/contracts/State.sol#L160-L174](#)

Description:

As part of the management of the privacy pool state, the **_isKnownRoot** function uses a circular buffer to store the most recent **ROOT_HISTORY_SIZE** roots (which is set to 30).

A circular buffer is a fixed-size array that continuously overwrites old data when new data is added, using an index that wraps around when reaching the end, meaning that if more than 30 deposits are made before a proof is submitted on-chain, the root of the oldest deposit will be overwritten.

If an attacker makes more than **ROOT_HISTORY_SIZE** deposits before a user submits their withdrawal proof, the root they rely on gets overwritten, effectively blocking the user from accessing their funds.

Recommendations:

- Ensure spamming is either prevented or difficult to achieve, for example, through one of the two methods:
 - Implement time or block-based delay for users between deposits & withdrawals.
 - Implement a time-based expiry mechanism, for example, store root/timestamp pairs and make old roots expire (via a predefined time window) only if they are older than the allowed time window - then have **_isKnownRoot** check both the root's existence and whether it is still within the valid time range.

Mitigation:

<https://github.com/0xbow-io/privacy-pools-core/blob/ac36feba8ba26f3485acd0d87a9d5c373ab3967d/packages/contracts/src/contracts/State.sol#L35>



AW-M-03: Unaudited Poseidon Hash Function Library

Severity: *Medium*

Status: *Acknowledged*

Code:

- [packages/contracts/src/contracts/PrivacyPool.sol#L19-L19](#)

Description:

During the audit it was found that the **poseidon-solidity** library is being used, which implements the Poseidon hash function in Solidity over the alt_bn128 (BN254) curve.

Poseidon is a ZKP-friendly hash function commonly used in zero-knowledge proof (ZKP) applications due to its efficiency in generating constraints.

However, in the [library repo](#), they state “**This implementation has not been audited**”.

Given this statement, it’s unreliable to depend on it for operational use as it can possibly lead to vulnerabilities such collision attacks, preimage attacks, logical issues and much more.

Recommendations:

- Investigate where there is a possible alternative repository fulfilling the specific protocol needs, and if it’s audited, prefer to use that one instead.
- If no better alternative is found, consider adding extra focus on increasing the verbosity of the tests done to logic related to this library (e.g. **PropertiesPoseidon.t.sol**), to test for boundary edge cases handling, collision resistance, randomized inputs etc.



AW-L-01: DoS Risks in ASP Root Management and Withdrawal Processing

Severity: *Medium*

Status: *No Risk (by design)*

Code:

- [packages/contracts/src/contracts/Entrypoint.sol#L90-L100](#)
- [packages/contracts/src/contracts/Entrypoint.sol#L129-L167](#)

Description:

The **updateRoot** function allows the **ASP_POSTMAN** to update the root by pushing a new **AssociationSetData** to the **associationSets** array. Each **AssociationSetData** contains the new root, an IPFS hash, and a timestamp. When a user submits a withdrawal proof, the protocol verifies that the proof's ASP root matches the latest root stored in the Entrypoint.

This unintentionally gives **ASP_POSTMAN** the power to continuously update the root, invalidating all pending withdrawal proofs (that were valid before the update), as they no longer match the latest root.

Moreover, in the relay function, after a withdrawal is processed, the funds are transferred to both the recipient and the fee recipient specified in **RelayData**. If either of these transfers fails (e.g., the recipient is a contract that reverts on receiving funds), the entire withdrawal transaction will revert, preventing users from being able to withdraw their funds.

Both cases can lead to DoS and preventing users from withdrawals, effectively blocking users from being able to access their funds.

Recommendations:

- Introduce a time-based restriction on how often the **ASP_POSTMAN** can update the root, or alternatively, use a decentralized entity to perform the update.
- On the **relay** function, use a try-catch block to handle the transfer failures gracefully. This way, if the transfer to the recipient or fee recipient fails, the transaction will not revert, and you can handle the failure accordingly.
 - Alternatively, require relayers to withdraw their fees in a separate transaction.



AW-L-02: Incorrect Index Returned in updateRoot Function

Severity: *Low*

Status: *Mitigated*

Code:

- [packages/contracts/src/contracts/Entrypoint.sol#L96-L97](#)

Description:

The **updateRoot** function in the Entrypoint contract incorrectly calculates the index of the newly added element in the **associationSets** array.

The function returns `associationSets.length`, which is one greater than the last valid index of the array, resulting in an incorrect and out-of-bounds index being returned from the function.

This means that any logic using **updateRoot** and relying on its return value is receiving an incorrect value, that is out-of-bounds of the array. It could lead to inconsistencies across the codebase and unexpected issues.

Recommendations:

- Modify the function to return `associationSets.length - 1` to ensure the correct zero-based index is returned.

Mitigation:

<https://github.com/0xbow-io/privacy-pools-core/blob/ac36feba8ba26f3485acd0d87a9d5c373ab3967d/packages/contracts/src/contracts/Entrypoint.sol#L97>



AW-L-03: Unbounded Relay Fee

Severity: *Low*

Status: *Mitigated*

Code:

- [packages/contracts/src/contracts/Entrypoint.sol#L201-L201](#)

Description:

The relay function in the smart contract lacks proper validation of the **relayFeeBPS** value. Although the **_deductFee** function ensures that the fee cannot be 100% or more, there is no upper limit enforced on the fee percentage within the relay function itself.

This oversight allows a malicious relayer to set an excessively high fee (e.g., 99.99%), effectively siphoning off the majority of the withdrawn funds.

This poses a risk to users who rely on the relay service, as their funds could be substantially depleted without their knowledge or consent.

Recommendations:

- Define a maximum allowable fee percentage (e.g., 5% or 10%) that can be charged to users.

Mitigation:

<https://github.com/0xbow-io/privacy-pools-core/blob/6cdeeaf0e44b98d5563c9d2f340f5825bec117f2/packages/contracts/src/contracts/Entrypoint.sol#L153>





This report was produced by Auditware for Oxbow based solely on the information provided by Oxbow. Auditware provides no guarantees as to the accuracy of the contents of this report and does not make any guarantees that following the advice within will prevent security incidents or issues.

Auditware is available for questions or comments about any of the contents of this report. We can be reached at <https://auditware.io/> or by email at joe@auditware.io.

